MOBIUS TENSORS

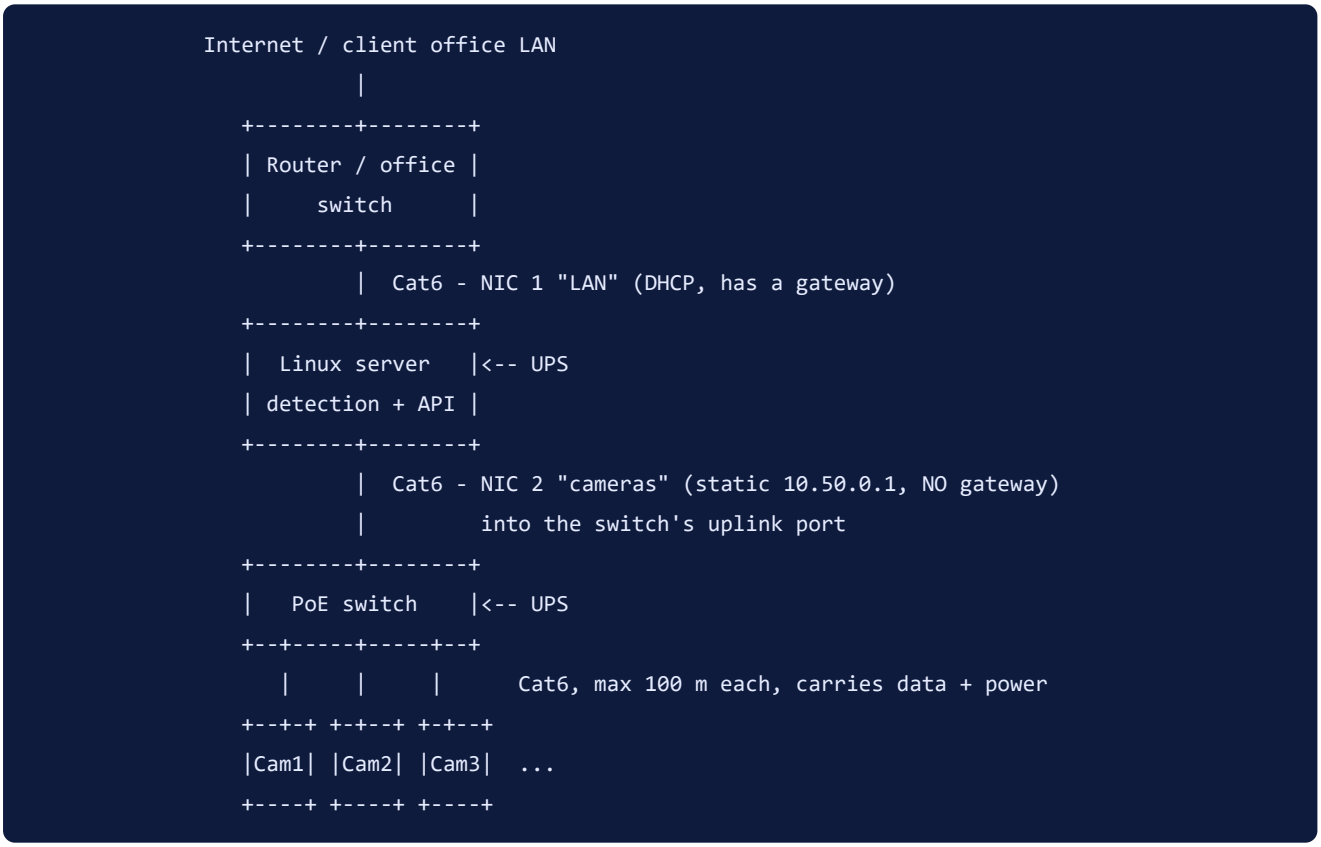
Site Installation Guide — IP Cameras, PoE Switch and Server

2026-10-05 · Mobius Tensors

This is the step-by-step guide for installing a Mobius Tensors system at a client site: cabling the cameras to a PoE switch, connecting the switch to the server, putting everything on the right network, and starting the detection software. Follow it in order. Every step ends with a check, so a problem is caught where it happens instead of three steps later.

It matches the architecture in *architecture.md*: cameras live on an **isolated camera network**, the server is the only machine that talks to them, and nothing about the cameras is exposed to the internet.

1. What you are building



- **PoE switch**: one cable per camera carries both video and power. No power supply at the camera.
- **Server, two network ports**: one port faces the cameras, the other faces the office LAN / internet. The server does **not** route between them, so cameras can never reach the internet and the internet can never reach the cameras.
- **Software**: one detection process per camera reads its RTSP stream, plus one API process that serves events, snapshots and search.

> **Only one network port on the server?** Add a USB 3.0 gigabit Ethernet adapter (cheap and reliable) and use it as the camera port. The alternative, a managed switch with VLANs, works too but takes more configuration and is easier to get wrong.

2. Bill of materials

Item	What to buy	Notes
IP cameras	ONVIF + RTSP, H.264 (or H.265), PoE (802.3af)	Hikvision, Dahua, Reolink, Uniview and Axis have ready-made settings in our config. Avoid "cloud-only" / Wi-Fi consumer cameras: they often have no RTSP.
PoE switch	Ports ≥ cameras + 1 (uplink) + 1 spare. **Gigabit uplink.** 802.3af/at	Check the **total PoE budget** (section 3.2), not just the port count. Unmanaged is fine with a 2-port server.
Server	x86-64, 4+ cores, 16 GB RAM, SSD for the system, **a large HDD for recordings**, **2 Ethernet ports**	Ubuntu Server 24.04 LTS. CPU-only is fine to start (see the scaling note in *architecture.md*).
Cable	Cat6 (Cat5e acceptable), **solid pure copper**, outdoor/UV-rated outside	Never CCA (copper-clad aluminium): it overheats and drops voltage under PoE.
Connectors	RJ45 plugs (shielded if the cable is shielded), boots	Plus outdoor junction boxes / waterproof caps for the camera end.
Recording disk	Surveillance-grade HDD (WD Purple, Seagate SkyHawk)	1080p at 4 Mbit/s = **~43 GB per camera per day**. 8 cameras × 7 days ≈ 2.4 TB.
Tools	Crimper, cable tester (one that shows the wire map), ladder, drill	A PoE tester is a cheap, big time saver.
Power	UPS sized for server + switch	If the switch loses power, every camera goes dark.
Commissioning laptop	Any laptop with an Ethernet port	Used to activate cameras with the vendor's tool (Hikvision SADP, Dahua ConfigTool).

3. Plan before you drill

3.1 IP address plan

Write this down before you touch a camera, and leave a copy at the site.

Device	IP address	Notes
Server, camera port (NIC 2)	`10.50.0.1/24`	No gateway
Camera 1 (e.g. `gate`)	`10.50.0.11`	Gateway `10.50.0.1` or empty
Camera 2 (e.g. `yard`)	`10.50.0.12`	
Camera 3 (e.g. `shop_entrance`)	`10.50.0.13`	
Managed switch (if any)	`10.50.0.2`	
Server, LAN port (NIC 1)	From the client's DHCP	Ask IT to reserve it

Why `10.50.0.0/24` and not the camera factory default `192.168.1.x`: many offices already use `192.168.1.0/24`. If the camera network uses the same range as the office LAN, the server can't tell which port to send traffic to. A distinct range avoids that. If `10.50.0.0/24` happens to be in use at the client, pick any other private range that isn't.

3.2 PoE power budget

Add up the **maximum** power of every camera (from its datasheet, "max power consumption"), with IR night lights on. That is the load the switch must deliver.

Standard	Per port at the switch	Typical device
802.3af (PoE)	15.4 W	Fixed bullet / dome camera
802.3at (PoE+)	30 W	Cameras with heater, motorised zoom, strong IR
802.3bt (PoE++)	60–90 W	PTZ cameras with heater

Example: 6 cameras × 7 W = 42 W. A switch with a 65 W budget is fine; one with a 30 W budget looks fine during the day and starts rebooting cameras at night when the IR LEDs switch on. Keep **at least 20 % headroom**.

3.3 Cable runs

- **100 m maximum** per cable, switch to camera, including patch cords. Longer runs need a PoE extender or a second switch closer to the cameras.
- Keep data cables at least 30 cm away from mains power cables when running in parallel; cross them at right angles.
- Outdoor runs: outdoor-rated cable in conduit, a drip loop before each entry point, and a surge protector for long exposed runs.

3.4 Bandwidth

Stream	Typical bitrate (H.264)
Sub stream (640×360 – 704×576)	0.5 – 1 Mbit/s
Main stream 1080p	4 – 6 Mbit/s
Main stream 4 MP / 4K	6 – 16 Mbit/s

The detector uses the **sub stream** by default, so even 20 cameras stay well under 100 Mbit/s. The server also opens every camera's **main stream** (recording + hi-res crops for search), so use a **gigabit** uplink: 20 cameras × 6 Mbit/s main + 1 Mbit/s sub is about 140 Mbit/s.

4. Physical installation

Step 1 — Terminate and test every cable

1. Crimp both ends with the **T568B** standard (same on both ends).
2. Test each cable with the cable tester **before** mounting anything. All 8 wires must pass.
3. Label both ends with the camera name (`gate`, `yard`…) and the switch port number.

Check: every cable passes the tester, every end is labelled.

Step 2 — Mount the cameras

1. Mount at 2.5–4 m height, angled down. For people, faces need ~32+ pixels high on the image to be usable by face search: that means a camera close to the door, not a wide shot of the car park.
2. Avoid pointing straight into the sun, headlights or bright windows (backlight ruins detection).
3. Seal the outdoor connector (waterproof cap or junction box).

Step 3 — Install the switch and server

1. Place the server and the PoE switch together in a ventilated cabinet or rack, both on the UPS.
2. Plug each camera cable into a PoE port of the switch. Write down which port is which camera.
3. Connect the **switch's uplink port** to the server's **NIC 2** (camera port).
4. Connect the server's **NIC 1** to the client's LAN (router or office switch).
5. Power on the switch, then the server.

Check: on the switch, each camera port shows a link LED and a PoE LED. The camera's IR LEDs glow faintly red in the dark after ~1 minute. If a port has link but no PoE, check the PoE budget and the cable.

> **Important:** at this point **do not connect the PoE switch to the office LAN as well.** The camera network must stay connected to the server only.

5. First-time camera setup (activation)

New cameras come out of the box with the **same factory IP** (Hikvision `192.168.1.64`, Dahua `192.168.1.108`, Reolink: DHCP) and, for most vendors, **no password** until they are activated. Two cameras with the same IP on the same switch conflict with each other, so configure them **one at a time**.

The easiest way is with the commissioning laptop plugged directly into a spare port of the PoE switch.

Step 4 — Put the laptop on the factory subnet

In an **admin PowerShell** (`Get-NetAdapter` shows the adapter name):

```
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.1.10 -PrefixLength 24
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 10.50.0.250 -PrefixLength 24
```

The laptop now talks both to factory-default cameras and to already-configured ones. Don't set a gateway, so the laptop's internet keeps going over Wi-Fi.

Step 5 — Activate and re-address each camera

Unplug all cameras from the switch except one, then for each camera:

1. Run the vendor tool (**Hikvision SADP**, **Dahua ConfigTool**) or open `http://192.168.1.64` (or `.108`) in a browser.
2. **Activate** it with a strong, unique password. Save it in the company password manager under the site and camera name. Never leave a default or shared password.
3. Change the camera's IP to the one in your plan (e.g. `10.50.0.11`, mask `255.255.255.0`, gateway `10.50.0.1`).
4. Open the camera's web page at its new address and set:

Setting	Value	Why
ONVIF	**Enabled**, create an ONVIF user	Lets `cameras.py discover` find it
RTSP	Enabled, port **554**, authentication **digest**	How the server reads video
Sub stream	H.264, 640×360 to 704×576, **10–15 FPS**, I-frame interval = 2× FPS	What the detector reads: cheap to decode on CPU
Main stream	H.264 (H.265 only if the server decodes it fine), 10–15 FPS, **I-frame interval = FPS** (one keyframe per second), constant or capped bitrate	Recorded as-is; its keyframes give the full-resolution crops used by search and face search
Smart codec (H.264+ / H.265+)	**Off**	Long I-frame gaps cause grey/smeared frames after reconnects
Time	NTP server `10.50.0.1`, correct time zone	Event times and searches like "yesterday 6pm" depend on it
UPnP, P2P / cloud (Hik-Connect, DMSS…)	**Off**	The camera must not try to reach the internet
Firmware	Latest from the vendor's site	Many older firmwares have known vulnerabilities
On-screen camera name	Same as in the config (`gate`, `yard`…)	Makes snapshots self-explanatory

Then plug in the next camera and repeat.

Check: all cameras are plugged in again, and each one opens in the browser at its planned `10.50.0.x` address.

> **Client already has an NVR?** You can read the cameras through the NVR instead: use the NVR's IP and the `channel:` setting in the config. Direct-to-camera is still preferred: one less box that can fail and the stream isn't re-encoded.

6. Server setup

Ubuntu Server 24.04 LTS, freshly installed, with a non-root admin user and SSH enabled.

Step 6 — Configure the two network ports

Find the port names:

```
ip -br link      # e.g. enp1s0 (to LAN) and enp2s0 (to PoE switch)
```

Unplug one cable and run it again if you're not sure which is which: the one that shows `DOWN` is the one you unplugged.

Create `/etc/netplan/60-mobius.yaml` (replace the port names):

```
network:
  version: 2
  ethernets:
    enp1s0:                    # NIC 1: client LAN / internet
      dhcp4: true
    enp2s0:                    # NIC 2: PoE switch / cameras
      dhcp4: false
      addresses: [10.50.0.1/24]
      # no gateway, no DNS: this network goes nowhere
```

Apply it:

```
sudo chmod 600 /etc/netplan/60-mobius.yaml
sudo netplan try        # press Enter within 120 s to keep it, otherwise it rolls back
```

Check:

```
ip -br addr show enp2s0       # shows 10.50.0.1/24
ping -c 3 10.50.0.11          # every camera answers
ip route                      # exactly one "default via" line, on enp1s0
```

Step 7 — Keep the networks separate

Make sure the server never forwards packets between the two networks (Ubuntu's default, but check):

```
sysctl net.ipv4.ip_forward    # must print 0
```

Turn on the firewall: allow SSH from the LAN, allow the cameras to get the time, block everything else coming in.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow in on enp1s0 to any port 22 proto tcp
sudo ufw allow in on enp2s0 from 10.50.0.0/24 to any port 123 proto udp
sudo ufw enable
```

Never port-forward RTSP (554) or the camera web pages (80/443) on the client's router.

Step 8 — Serve the time to the cameras

```
sudo apt install -y chrony
echo "allow 10.50.0.0/24" | sudo tee /etc/chrony/conf.d/cameras.conf
sudo systemctl restart chrony
timedatectl                   # "System clock synchronized: yes", correct time zone
```

Set the server's time zone if needed: `sudo timedatectl set-timezone Asia/Beirut` (or the site's zone).

Check: after a few minutes the camera's clock (web page → Time) matches the server.

7. Install the software

Step 9 — Install uv and the code

```
sudo apt install -y git ffmpeg
curl -LsSf https://astral.sh/uv/install.sh | sh     # installs uv to ~/.local/bin
sudo mkdir -p /opt/mobius && sudo chown $USER /opt/mobius
git clone <repo-url> /opt/mobius/mobius-tensors
cd /opt/mobius/mobius-tensors/projects/object-detection
uv sync --group search --group server
uv run src/download_model.py yolox_nano
uv run --group export src/export_clip.py          # only if text search is used (~600 MB)
uv run src/download_model.py faces                # only if face search is used AND approved
```

Step 10 — Find the cameras from the server

```
uv run src/cameras.py discover                    # ONVIF: lists IP + model of each camera
uv run src/cameras.py scan 10.50.0.0/24           # fallback: every host with RTSP (554) open
```

Every camera in your IP plan must appear. If `discover` finds nothing but `scan` does, ONVIF is off on the camera (that's fine, RTSP is what matters).

Step 11 — Write the site config

Copy the example and name it after the site:

```
cp configs/search_example.yaml configs/site.yaml
```

Declare the cameras in `configs/site.yaml`:

```
cameras:
  gate:
    vendor: hikvision          # hikvision | dahua | reolink | uniview | axis
    host: 10.50.0.11
    username: admin
    password: ${GATE_CAM_PASSWORD}
    stream: sub                # low-res sub stream: enough for detection, far cheaper on CPU
  yard:
    vendor: dahua
    host: 10.50.0.12
    username: admin
    password: ${YARD_CAM_PASSWORD}
    stream: sub
  other_brand:                 # unknown vendor: give the full RTSP URL
    url: rtsp://${OTHER_USER}:${OTHER_PASS}@10.50.0.13:554/stream2
```

Put the passwords in `.env` next to the config (git-ignored), never in the YAML:

```
cat > .env <<'EOF'
GATE_CAM_PASSWORD=...
YARD_CAM_PASSWORD=...
EOF
chmod 600 .env
```

Point the recordings at the big disk (mounted e.g. at `/srv/recordings`) and agree the retention with the client:

```
recording:
  enabled: true
  dir: /srv/recordings          # one MP4 per camera per minute, under <dir>/<camera>/
  retention_days: 7
  min_free_gb: 50               # delete the oldest files first if the disk fills up
  hires_crops: true             # search/face crops from main-stream keyframes
```

Then add the site's zones, lines and alert rules (draw them in the test UI's **Zones & lines** tab and paste the YAML; see the *object-detection README*).

Step 12 — Test each camera

```
uv run src/cameras.py --config configs/site.yaml list                    # URLs, passwords masked
uv run src/cameras.py --config configs/site.yaml test gate --save data/gate.jpg
```

Expected: `OK: 640x360, ~15.0 FPS received` and a snapshot in `data/gate.jpg`. Copy the snapshot to your laptop (`scp`) and look at it: right camera, right angle, in focus, not upside down.

Then run the full pipeline once in the foreground and watch for detections and events:

```
uv run --group search src/main.py --config configs/site.yaml --source gate
```

Stop it with `Ctrl+C` once it works.

8. Run it as a service

The system must start on its own after a power cut and restart a process that crashes. One systemd service per camera, plus one for the API.

Step 13 — Create the services

`/etc/systemd/system/mobius-camera@.service` (the `@` makes it a template: one instance per camera name):

```
[Unit]
Description=Mobius Tensors detection, camera %i
After=network-online.target
Wants=network-online.target

[Service]
User=mobius
WorkingDirectory=/opt/mobius/mobius-tensors/projects/object-detection
ExecStart=/home/mobius/.local/bin/uv run --group search src/main.py --config configs/site.yaml --sou
rce %i
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

`/etc/systemd/system/mobius-api.service`:

```
[Unit]
Description=Mobius Tensors API
After=network-online.target

[Service]
User=mobius
WorkingDirectory=/opt/mobius/mobius-tensors/projects/object-detection
ExecStart=/home/mobius/.local/bin/uv run --group server --group search src/server.py --config config
s/site.yaml
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Replace `mobius` with the server's user name. Then enable one instance per camera:

```
sudo systemctl daemon-reload
sudo systemctl enable --now mobius-camera@gate mobius-camera@yard mobius-api
systemctl status 'mobius-*'
journalctl -u mobius-camera@gate -f              # live logs of one camera
```

> **Histor (search/chat over everything seen)** uses the same camera config. To record for Histor instead, point `ExecStart` at `projects/histor` and `src/record.py`. See the *Histor README*.

Check: reboot the server (`sudo reboot`). Without logging in, every service comes back up. Unplug a camera for 30 seconds and plug it back: its log shows the stream reconnecting by itself.

9. Access the dashboard and API safely

The API listens on `127.0.0.1:8000` only, and it has **no login yet**. Do **not** change `--host` to `0.0.0.0` on a site server.

From your laptop, open an SSH tunnel and use the API as if it were local:

```
ssh -L 8000:127.0.0.1:8000 mobius@<server-LAN-IP>
```

Then open http://127.0.0.1:8000/docs on the laptop. For permanent remote access, use a VPN (WireGuard / Tailscale) or a reverse proxy with authentication and HTTPS in front of the API, never a raw port forward.

Check: `http://127.0.0.1:8000/health` returns counts, `/cameras` lists every camera, and `/events` fills up as people/vehicles move through the zones.

10. Security and privacy checklist

- ☐ Every camera has its own strong password; no factory defaults remain.
- ☐ Camera P2P/cloud and UPnP are off; camera firmware is up to date.
- ☐ The PoE switch is connected to the server only, never to the office LAN.
- ☐ `net.ipv4.ip_forward = 0`; the camera port has no gateway.
- ☐ Firewall on; no router port forwards for cameras or the API.
- ☐ API bound to `127.0.0.1`, reached through SSH tunnel / VPN / authenticated proxy.
- ☐ `.env` is `chmod 600` and not committed anywhere.
- ☐ Face search: only enabled with the client's **written approval**, local law checked, camera notices posted, `face_retention_days` set (default 7).
- ☐ `retention_days` (events/search) and `recording.retention_days` (video) agreed with the client and set in the config.

11. Commissioning checklist (hand-over)

- ☐ IP plan and switch port map printed and left in the cabinet.
- ☐ Every cable tested and labelled at both ends.
- ☐ PoE budget checked at night with all IR lights on (no camera reboots).
- ☐ All cameras show the correct time (NTP from the server).
- ☐ `cameras.py test` passes for every camera; snapshots reviewed.
- ☐ Services survive a reboot and a camera unplug/replug.
- ☐ Walk test: someone walks each zone / crosses each line, and the events appear in `/events` with a correct snapshot, and alerts (webhook / email) arrive.
- ☐ Server and switch on UPS; UPS tested by pulling the mains plug.
- ☐ Passwords stored in the company password manager under the site name.

12. Troubleshooting

Symptom	Likely cause	Fix
Switch port: link LED but no PoE LED	Budget exceeded, or camera needs PoE+	Check total budget; move it to a PoE+ port
Camera reboots at night	IR LEDs raise power draw over the budget	Bigger switch or fewer cameras per switch
No link at all	Bad crimp, cable > 100 m, CCA cable	Re-test the cable; re-crimp; replace with pure copper
Server's camera port shows `169.254.x.x`	Netplan not applied to the right port	Fix port name in `/etc/netplan/60-mobius.yaml`, `sudo netplan apply`
`ping` works, `cameras.py test` fails	Wrong RTSP path, password, or RTSP disabled	Try the URL in VLC from the laptop; check `vendor:` / `path:`; re-type the password in `.env`
`401 Unauthorized`	Wrong password, or camera locked after failed logins	Wait 30 min or unlock from the camera web page; fix `.env`
`discover` finds nothing, `scan` finds the cameras	ONVIF off	Enable ONVIF on the camera (optional)
Two cameras "flicker" in and out	Duplicate IP (two cameras still on factory default)	Re-do Step 5 with one camera plugged in at a time
Grey / smeared frames	Smart codec on, or packet loss	Turn off H.264+/H.265+; check cable; streams already use TCP
Events show the wrong time	Camera or server clock wrong	Check `timedatectl`, chrony, camera NTP setting
Log says `ffmpeg not found`	ffmpeg missing	`sudo apt install ffmpeg`, restart the services
No files in the recordings folder, `Could not find tag for codec`	Camera main stream is MJPEG or an unusual codec	Set the main stream to H.264
Search results have `video_url: null`	No recording at that time (not enabled, disk full, camera was off)	Check `recording.enabled`, free space, `journalctl -u mobius-camera@<name>`
CPU at 100 %, detection lags	Too many cameras or main stream used	Use `stream: sub`, lower FPS on the camera, smaller model; then see the CPU vs GPU decision in *architecture.md*
Office internet breaks when the server is plugged in	A gateway was set on the camera port, or the PoE switch was also cabled to the LAN	Remove the gateway; disconnect the switch from the LAN

Appendix A — RTSP paths by vendor

What `vendor:` expands to in the config (`channel` = 1 for a standalone camera, `N` for an NVR channel):

Vendor	Main stream	Sub stream
Hikvision	`/Streaming/Channels/101`	`/Streaming/Channels/102`

Vendor	Main stream	Sub stream
Dahua	`/cam/realmonitor?channel=1&subtype=0`	`/cam/realmonitor?channel=1&subtype=1`
Reolink	`/h264Preview_01_main`	`/h264Preview_01_sub`
Uniview	`/unicast/c1/s0/live`	`/unicast/c1/s1/live`
Axis	`/axis-media/media.amp?camera=1`	(same, set resolution in the camera)

Full URL form: `rtsp://user:password@10.50.0.11:554/<path>`. Test any URL from a laptop with VLC (*Media → Open Network Stream*) or from the server with:

```
ffprobe -rtsp_transport tcp "rtsp://user:password@10.50.0.11:554/Streaming/Channels/102"
```

Appendix B — Sizing rule of thumb (CPU server)

Cameras	Server	Model	Recording disk (1080p, 7 days)
1 – 4	4-core CPU, 16 GB RAM	`yolox_nano` / `yolox_tiny`, sub stream	2 TB
5 – 12	8-core CPU, 32 GB RAM	`yolox_tiny`, sub stream, 10 FPS	4 – 6 TB
12+ / heavy text search	Benchmark first; GPU server if the CPU can't keep up	See *architecture.md § 10*	~0.3 TB per camera

These are starting points, not guarantees: always run the site's cameras for an hour and watch CPU (`htop`) before signing off.